Дашборды компании

ПОКАЗАТЕЛИ СЛУЖБ АЛМИ ПАРТНЕР

Руководство по развёртыванию

Версия 0.4.0

Ноябрь 2026

© Alexander Yurasov

Содержание

1 О документе 4

2 Архитектура 5

3 Требования 7

3.1 Сервер 7

3.2 Клиенты 7

3.3 Сборка из исходного кода 7

4 Установка 8

4.1 Шаг 1. Получение дистрибутива 8

4.2 Шаг 2. Конфигурация 8

4.3 Шаг 3. Сборка и запуск 9

4.4 Шаг 4. Проверка 9

4.5 Шаг 5. HTTPS 9

4.6 Шаг 6. Первый вход 10

5 Демонстрационный стенд 11

6 Переменные окружения 12

6.1 Основные 12

6.2 Сессии и безопасность 13

6.3 Вход по домену (LDAP) 13

6.4 Синхронизация с Active Directory 15

6.5 Входящие данные и интеграции 16

6.6 Производительность 18

6.7 Журнал 19

7 Вход по домену и Active Directory 20

7.1 Настройка входа по домену 20

7.2 Синхронизация по расписанию 21

8 Входящие данные и интеграции 24

8.1 Планировщик (worker) 24

8.2 Папка «Входящие» 25

8.3 Сетевой доступ к системам 26

8.4 Секреты подключений и SECRET_KEY 27

8.5 Exchange и exchangelib 27

8.6 Без планировщика: cron 28

9 База данных и миграции 30

10 Резервное копирование и восстановление 32

10.1 PostgreSQL 32

10.2 SQLite 32

11 Обновление 33
11.1 Обычное обновление 33
11.2 Обновление до 0.4.0 33
11.3 Обновление до 0.3.0 35
11.4 Обновление до 0.2.0 37
11.5 Обновление до 0.1.0 с версий до 0.1.0 38
12 Производительность и масштабирование 40
13 Обслуживание 41
14 Диагностика 43

1 О документе

Руководство описывает установку, настройку, обновление и обслуживание «Дашбордов компании» версии 0.4.0: состав системы, требования к серверу, запуск в Docker, переменные окружения (в том числе часовой пояс, вход по учётной записи домена, синхронизацию с Active Directory, входящие данные и интеграции), планировщик, папку «Входящие», сетевой доступ к системам компании и шифрование секретов подключений, миграции базы данных, резервное копирование, обновление до 0.4.0, 0.3.0 и 0.2.0, проверку производительности и диагностику.

Документ предназначен для системных администраторов и инженеров эксплуатации. Работа с интерфейсом описана в «Руководстве пользователя», администрирование учётных записей, доступа, структуры компании и служб — в «Руководстве администратора».

Условные обозначения

ОБОЗНАЧЕНИЕ	ЗНАЧЕНИЕ
`команда`	Команда, имя файла или переменной окружения
> **Важно.**	Правило, нарушение которого приводит к ошибке или потере данных
> **Совет.**	Рекомендация

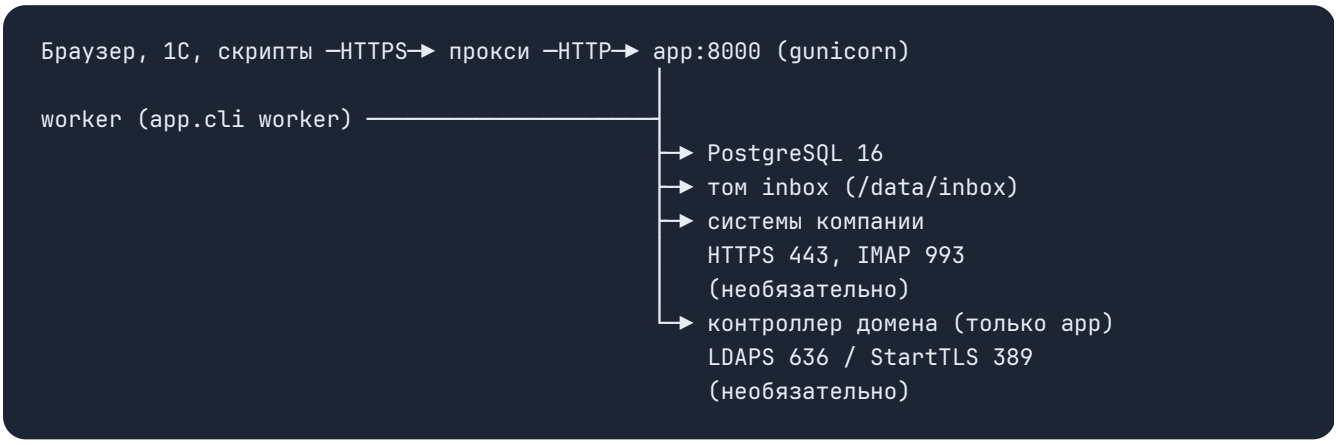
2 Архитектура

Система состоит из трёх компонентов, которые поставляются одним Docker-образом, и базы данных; каталог домена и системы компании подключаются по желанию:

КОМПОНЕНТ	ТЕХНОЛОГИИ	НАЗНАЧЕНИЕ
Интерфейс	Vue 3, Vite, Pinia, Vue Router, Apache ECharts	Одностраничное приложение; собирается при сборке образа и раздаётся сервером
Сервер	Python 3.12, FastAPI, SQLAlchemy 2, Alembic, gunicorn + uvicorn	API, проверка прав, расчёт аналитики и сквозного отчёта, структура компании, PDF-отчёты (ReportLab, Matplotlib), импорт и экспорт Excel (openpyxl), вход по домену и синхронизация с AD (ldap3), входящие данные и приём файлов по API, подключения к системам компании (requests, cryptography, exchangelib)
Планировщик	тот же образ, `python -m app.cli worker`	Разбор папки «Входящие», запуски подключений по расписанию, очистка по сроку хранения; HTTP-порта нет
База данных	PostgreSQL 16 (рекомендуется) или SQLite	Данные служб, структура компании, пользователи и доступ, сессии, журнал изменений, входящие данные с исходными файлами до срока хранения, подключения с зашифрованными секретами
Каталог (необязательно)	Active Directory, FreeIPA, OpenLDAP	Вход по учётной записи домена, синхронизация подразделений и сотрудников
Системы компании (необязательно)	«По Делу», «Пульс проекта», Jira, Confluence, Bitrix24, Exchange / Microsoft 365 / IMAP, 1С (OData)	Структура и сотрудники, показатели сквозного отчёта, файлы во «Входящие», публикация отчёта

Запросы обслуживают несколько рабочих процессов gunicorn с воркерами uvicorn. Состояние хранится только в базе данных (включая сессии), поэтому процессы и экземпляры приложения взаимозаменяемы: их число можно увеличивать без настройки «липких» сессий.

Фоновую работу — папку «Входящие» и расписания подключений — выполняет отдельный процесс-планировщик (сервис `worker`) с той же базой и тем же томом папки; он не принимает запросов и может работать в нескольких экземплярах. Снимок структуры компании каждый процесс держит в памяти и перестраивает только при изменении структуры (по её ревизии в базе). Так же сквозной отчёт за месяц рассчитывается один раз на набор ревизий — структуры, данных служб и значений показателей — и затем отбирается под права каждого пользователя без пересчёта.



3 Требования

3.1 Сервер

ПАРАМЕТР	МИНИМУМ	РЕКОМЕНДУЕТСЯ ДЛЯ 500–1000 ПОЛЬЗОВАТЕЛЕЙ
Процессор	2 ядра	4 ядра и более
Память	2 ГБ	4–8 ГБ
Диск	10 ГБ	20 ГБ SSD
ОС	Linux x86_64 с Docker 24+ и Docker Compose v2	Ubuntu 24.04 LTS / Astra Linux / РЕД ОС

Отдельный сервер PostgreSQL не обязателен: `docker-compose.yml` запускает его в соседнем контейнере. Для промышленной эксплуатации можно использовать внешний кластер PostgreSQL 14+.

Для входа по домену и синхронизации с Active Directory серверу приложения нужен доступ к контроллеру домена по LDAPS (порт 636) или LDAP со StartTLS (порт 389), служебная учётная запись с правом чтения каталога и сертификат центра сертификации домена, если он не доверенный в системе.

Для интеграций контейнерам `app` и `worker` нужен исходящий доступ к системам компании — напрямую или через прокси `HTTPS_PROXY` (раздел «Сетевой доступ к системам»), а для приёма файлов от 1С и скриптов — входящий HTTPS к приложению через обратный прокси. Исходные файлы входящих хранятся в базе до срока хранения (по умолчанию 90 дней) и в томе папки «Входящие»: заложите место по объёму выгрузок — при ежедневных файлах по 1–5 МБ это сотни мегабайт.

3.2 Клиенты

Современный браузер: Яндекс Браузер, Chrome, Edge, Firefox последних версий; экран от 1280 пикселей по ширине (на узких экранах и телефонах интерфейс перестраивается).

3.3 Сборка из исходного кода

Для разработки и сборки без Docker нужны Python 3.12+, Node.js 20.19+ (рекомендуется 22) и npm.

4 Установка

4.1 Шаг 1. Получение дистрибутива

```
git clone https://github.com/ayurasov/dashboards_serv.git
cd dashboards_serv
git checkout v0.4.0
```

4.2 Шаг 2. Конфигурация

```
cp .env.example .env
```

Откройте `.env` и задайте как минимум:

ПЕРЕМЕННАЯ	ЧТО УКАЗАТЬ
`SECRET_KEY`	Случайная строка не короче 32 символов: `openssl rand -hex 32`. Ключом шифруются секреты подключений к системам компании — сохраните его в надёжном месте: после смены ключа их придётся ввести заново
`POSTGRES_PASSWORD`	Пароль пользователя базы данных
`ADMIN_PASSWORD`	Пароль первого администратора (не короче 8 символов). Если оставить пустым, временный пароль будет выведен в журнал контейнера
`APP_PORT`	Порт приложения на сервере (по умолчанию 8000)
`TIMEZONE`	Часовой пояс компании, если он не московский (по умолчанию `Europe/Moscow`)
`PUBLIC_URL`	Адрес, по которому пользователи открывают систему (`https://dashboards.company.ru`), — для ссылок на страницах Confluence и в комментариях «По Делу»; можно задать позже

Вход по учётной записи домена и интеграции настраиваются позже, после первого входа (разделы «Вход по домену и Active Directory» и «Входящие данные и интеграции»).

> **Важно.** С ключом по умолчанию сервер в режиме production не запускается. Храните `.env` вне системы контроля версий.

4.3 Шаг 3. Сборка и запуск

```
docker compose up -d --build
docker compose logs -f app
```

Запускаются три контейнера: `db` (PostgreSQL), `app` (приложение) и `worker` (планировщик: папка «Входящие» и расписания подключений). При старте контейнер приложения:

1. ждёт готовности PostgreSQL;
2. применяет миграции базы данных (`python -m app.cli migrate`) — под блокировкой, поэтому одновременный старт нескольких контейнеров безопасен;
3. приводит справочники к актуальному виду (службы, метрики, правила светофора, палитры, встроенные профили доступа, типы уровней структуры);
4. создаёт первого администратора, если пользователей в базе нет;
5. запускает gunicorn.

Планировщик стартует после того, как приложение станет здоровым, ждёт актуальную схему базы (миграции выполняет только `app`) и пишет в журнал «Планировщик запущен: такт 30 с, папка «Входящие»: /data/inbox» (`docker compose logs worker`).

4.4 Шаг 4. Проверка

```
curl -s http://127.0.0.1:8000/api/health
```

Ответ `{"status": "ok", "version": "0.4.0", "database": "ok", "schema": {"current": "0016_integrations", "head": "0016_integrations"}}` означает, что сервер работает, база доступна и схема актуальна. Статус `degraded` (код 503) — схема базы не совпадает с версией приложения: проверьте журнал миграций.

Откройте `http://<сервер>:8000` и войдите под учётной записью администратора. Документация API (OpenAPI) доступна по адресу `/api/docs` . Планировщик проверяется на странице **Входящие данные → Папка**: блок «Планировщик» — «Работает» с отметкой не старше минуты, состояние папки — «Работает».

4.5 Шаг 5. HTTPS

Приложение рассчитано на работу за обратным прокси, который завершает TLS. Пример для Caddy (сертификат выпускается автоматически):

```
dashboards.company.ru {
    encode gzip
    reverse_proxy 127.0.0.1:8000
}
```

Пример для nginx:

```
server {
    listen 443 ssl http2;
    server_name dashboards.company.ru;
    ssl_certificate     /etc/ssl/dashboards.crt;
    ssl_certificate_key /etc/ssl/dashboards.key;
    client_max_body_size 25m;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }
}
```

После включения HTTPS задайте `SESSION_COOKIE_SECURE=true` и перезапустите приложение: cookie сессии будет передаваться только по защищённому соединению.

4.6 Шаг 6. Первый вход

1. Войдите под логином из `ADMIN_USERNAME` (по умолчанию `admin`) и паролем из `ADMIN_PASSWORD` или временным паролем из журнала (`docker compose logs app | grep администратор`).
2. Система попросит сменить пароль.
3. Постройте структуру компании и создайте учётные записи сотрудников с профилями доступа («Руководство администратора», разделы «Структура компании и подразделения», «Пользователи», «Импорт и экспорт»).
4. Заполните каталог показателей сквозного отчёта и выдайте права на их ввод («Руководство администратора», раздел «Показатели и сквозной отчёт»).
5. При необходимости включите вход по домену и синхронизацию с Active Directory (раздел «Вход по домену и Active Directory»).
6. Загрузите данные служб: недельный отчёт техподдержки и выгрузку Naumen — из файлов Excel, данные службы персонала — вводом по месяцам или файлом через «Входящие данные».
7. Подключите системы компании и выпустите ключи для систем, которые будут присылать файлы («Руководство администратора», раздел «Интеграции и входящие данные»; здесь — раздел «Входящие данные и интеграции»).

5 Демонстрационный стенд

Для показа и обучения можно запустить систему с демонстрационными данными: структура компании из 13 подразделений, девять учётных записей разных уровней (от генерального директора до инженера) с профилями доступа, семь месяцев данных службы персонала, 109 недель отчёта техподдержки, 1868 заявок Naumen, 142 партнёрства и каталог из 14 показателей сквозного отчёта со значениями за 12 месяцев.

```
DEMO_DATA=true docker compose up -d --build
```

Демонстрационные данные загружаются только в пустые разделы и никогда не перезаписывают существующие. Логины и пароли демо-стенда приведены в «Руководстве администратора» (раздел «Демонстрационный стенд»).

> **Важно.** Не включайте `DEMO_DATA` на рабочем сервере: демо-пользователи получают известные пароли.

6 Переменные окружения

`docker-compose.yml` передаёт контейнерам приложения и планировщика один и тот же набор переменных: из таблиц «Основные», «Сессии и безопасность», основные переменные входа по домену (`AUTH_LDAP_ENABLED` , `AUTH_LDAP_URL` , `AUTH_LDAP_BASE_DN` , `AUTH_LDAP_BIND_DN` , `AUTH_LDAP_BIND_PASSWORD`) и переменные входящих данных и интеграций, кроме `WORKER_TICK_SECONDS` , `INTEGRATION_RUNS_KEEP` и `HTTP_PROXY` . Остальные переменные добавьте в раздел `environment` сервисов `app` и `worker` — удобнее в файле `docker-compose.override.yml` , который Docker Compose подключает автоматически (пример — в разделе «Вход по домену и Active Directory»). Переменные интеграций нужны обоим сервисам: проверки и ручные запуски выполняет приложение, запуски по расписанию — планировщик.

6.1 Основные

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`APP_ENV`	`production` в образе	`production` — строгие проверки конфигурации; `development` — для разработки
`DATABASE_URL`	задаёт compose	Адрес базы: `postgresql+psycopg://user:pass@host:5432/db` или `sqlite:////data/dashboards.db`
`SECRET_KEY`	—	Секретный ключ сервера, не короче 32 символов; из него выводится ключ шифрования секретов подключений (раздел «Секреты подключений и SECRET_KEY»)
`ADMIN_USERNAME`	`admin`	Логин первого администратора
`ADMIN_PASSWORD`	пусто	Пароль первого администратора; пусто — временный пароль в журнале
`DEMO_DATA`	`false`	Загрузить демонстрационные данные в пустые разделы
`TIMEZONE`	`Europe/Moscow`	Часовой пояс компании (имя из базы IANA, например `Asia/Yekaterinburg`): границы дней в фильтре журнала изменений, время формирования в PDF-отчётах, даты, зависящие от «сегодня». Неизвестное имя — UTC
`CORS_ORIGINS`	пусто	Разрешённые источники через запятую — только если интерфейс открывается с другого адреса

6.2 Сессии и безопасность

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`SESSION_LIFETIME_HOURS`	`12`	Максимальная длительность сессии
`SESSION_IDLE_MINUTES`	`240`	Сессия завершается после простоя
`SESSION_COOKIE_SECURE`	`false`	`true` — cookie только по HTTPS
`PASSWORD_MIN_LENGTH`	`8`	Минимальная длина пароля
`LOGIN_RATE_LIMIT`	`10`	Неудачных попыток входа с одного адреса для одного логина за окно
`LOGIN_RATE_WINDOW_SECONDS`	`300`	Окно ограничения попыток входа

6.3 Вход по домену (LDAP)

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AUTH_LDAP_ENABLED`	`false`	`true` — включить вход по учётной записи домена
`AUTH_LDAP_URL`	пусто	Адрес контроллера домена: `ldaps://dc.company.local:636` (рекомендуется) или `ldap://dc.company.local:389` со StartTLS
`AUTH_LDAP_BASE_DN`	пусто	База поиска учётных записей, например `DC=company,DC=local`
`AUTH_LDAP_BIND_DN`	пусто	Служебная учётная запись только для чтения; пусто — анонимная привязка
`AUTH_LDAP_BIND_PASSWORD`	пусто	Пароль служебной учётной записи (в API и журнал не попадает)

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AUTH_LDAP_USER_FILTER`	`(&(objectClass=person)(|(sAMAccountName={login})(userPrincipalName={login})(mail={login})(uid={login})))`	Фильтр поиска; `{login}` — введённый логин (экранируется)
`AUTH_LDAP_ATTR_ID`	`objectGUID`	Неизменный идентификатор записи; FreeIPA и OpenLDAP — `entryUUID`
`AUTH_LDAP_ATTR_EMAIL`	`mail`	Атрибут e-mail — по нему учётные записи сопоставляются
`AUTH_LDAP_ATTR_NAME`	`displayName`	ФИО; FreeIPA — `cn`
`AUTH_LDAP_ATTR_LOGIN`	`sAMAccountName`	Логин для созданных учётных записей; FreeIPA — `uid`
`AUTH_LDAP_ATTR_POSITION`	`title`	Должность (заполняет пустую должность в системе)
`AUTH_LDAP_ATTR_GROUPS`	`memberOf`	Группы — для профиля доступа и группы администраторов
`AUTH_LDAP_CA_FILE`	пусто	Файл сертификата центра сертификации домена (PEM) внутри контейнера; для `ldap://` включает StartTLS
`AUTH_LDAP_START_TLS`	`false`	StartTLS для `ldap://` (для `ldaps://` не нужен)
`AUTH_LDAP_TIMEOUT`	`5`	Ожидание ответа каталога, с (1–30)
`AUTH_LDAP_CREATE_USERS`	пусто	Первый вход без учётной записи: пусто — отказ, `active` — создать активной, `inactive` — создать отключённой
`AUTH_LDAP_DEFAULT_PROFILE`	пусто	Ключ профиля доступа для созданных при входе (например, `employee`)
`AUTH_LDAP_GROUP_PROFILES`	пусто	Профили по группам: `DN группы=ключ профиля` через `;`, первое совпадение

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AUTH_LDAP_ADMIN_GROUP`	пусто	DN группы, участники которой создаются администраторами
`AUTH_LDAP_LOGIN_HINT`	пусто	Подсказка на странице входа; пусто — «Можно войти с логином и паролем домена — как при входе в Windows»

Сертификат сервера каталога проверяется всегда (своим файлом `AUTH_LDAP_CA_FILE` или системными корневыми сертификатами). Без `ldaps://` и StartTLS пароли идут по сети открытым текстом — страница «Вход по домену» предупредит об этом.

6.4 Синхронизация с Active Directory

Пустые адрес, база поиска, служебная учётная запись, сертификат и StartTLS берутся из настроек входа по домену; включать сам вход по домену (`AUTH_LDAP_ENABLED`) для синхронизации не обязательно.

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AD_SYNC_URL`	пусто → `AUTH_LDAP_URL`	Адрес каталога для синхронизации
`AD_SYNC_BASE_DN`	пусто → `AUTH_LDAP_BASE_DN`	База поиска сотрудников
`AD_SYNC_BIND_DN`	пусто → `AUTH_LDAP_BIND_DN`	Служебная учётная запись (берётся парой с паролем: своя или та же, что у входа)
`AD_SYNC_BIND_PASSWORD`	пусто → `AUTH_LDAP_BIND_PASSWORD`	Пароль служебной учётной записи
`AD_SYNC_USER_FILTER`	`(&(objectCategory=person)(objectClass=user)(mail=*))`	Какие записи считаются сотрудниками
`AD_SYNC_PATH_SOURCE`	`attributes`	Путь подразделения: `attributes` — из атрибутов, `ou` — из организационных единиц в DN
`AD_SYNC_PATH_ATTRIBUTES`	`company,division,department`	Атрибуты пути подразделения сверху вниз
`AD_SYNC_DETECT_HEADS`	`true`	Определять руководителей подразделений по атрибуту `manager` подчинённых

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AD_SYNC_CA_FILE`	пусто → `AUTH_LDAP_CA_FILE`	Сертификат центра сертификации
`AD_SYNC_START_TLS`	`false`	StartTLS для `ldap://`; без своего `AD_SYNC_URL` действует `AUTH_LDAP_START_TLS`

Ожидание ответа — `AUTH_LDAP_TIMEOUT`. Каталог читается постранично.

6.5 Входящие данные и интеграции

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`INBOX_DIR`	`/data/inbox` в compose; вне Docker — пусто	Папка «Входящие» на сервере; пусто — папка не опрашивается. В Docker — том `inbox`, общий для `app` и `worker`
`INBOX_SETTLE_SECONDS`	`10`	Файл в папке берётся, когда не меняется дольше стольких секунд (0–3600)
`INBOX_MAX_MB`	`20`	Предельный размер файла во входящих — из папки, загрузки и приёма по API (1–200)
`INBOX_RETENTION_DAYS`	`90`	Сколько дней после обработки хранятся исходные файлы — в базе и в `processed/`, `rejected/` папки (1–3650)
`INBOUND_RATE_LIMIT`	`120`	Запросов в минуту на один ключ приёма по API (`/api/inbound/v1`); сверх — 429 с `Retry-After`
`WORKER_TICK_SECONDS`	`30`	Такт планировщика: как часто он разбирает папку и проверяет расписания, с (1–3600)
`INTEGRATION_RUNS_KEEP`	`200`	Сколько запусков хранится в журнале каждого подключения (10–10000)

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`PUBLIC_URL`	пусто	Адрес, по которому пользователи открывают систему: `http(s)://узел[:порт][/путь]` без «/» в конце, логина, «?» и «#» — `https://dashboards.company.ru`. Нужен для ссылок наружу — «Открыть сквозной отчёт» на странице Confluence и «Входящие данные: .../inbox» в комментарии задачи «По Делу»; пусто — ссылки не ставятся. С неверным значением сервер не запустится («PUBLIC_URL — без «/» в конце: https://dashboards.company.ru»)
`HTTPS_PROXY`	пусто	Прокси для подключений к системам компании по `https://`: `http://proxy.company.local:3128`
`HTTP_PROXY`	пусто	Прокси для подключений по `http://` (в compose не передаётся — добавьте в override)
`NO_PROXY`	пусто	Узлы без прокси через запятую: `localhost,127.0.0.1,.company.local`
`PODELU_ALLOWED_HOSTS`	пусто	Разрешённые узлы подключений «По Делу»
`PULS_ALLOWED_HOSTS`	пусто	... «Пульса проекта»
`JIRA_ALLOWED_HOSTS`	пусто	... Jira
`CONFLUENCE_ALLOWED_HOSTS`	пусто	... Confluence
`BITRIX24_ALLOWED_HOSTS`	пусто	... Bitrix24
`EXCHANGE_ALLOWED_HOSTS`	пусто	... почтового ящика: сервер EWS, сервер IMAP, а для Microsoft 365 — `login.microsoftonline.com` и `graph.microsoft.com`
`ONEC_ALLOWED_HOSTS`	пусто	... 1С (OData)

Разрешённые узлы (`<ВИД>_ALLOWED_HOSTS`) — список через запятую: `jira.company.ru` — ровно этот узел, `.company.ru` — домен `company.ru` и все его поддомены. Пусто — подключение разрешено к любому узлу. Адрес подключения, не попадающий в список, не сохраняется и не

проверяется: «Подключение к … запрещено настройками сервера (JIRA_ALLOWED_HOSTS)». Задайте списки на рабочем сервере: так сотрудник с полномочием «Интеграции» не направит запросы сервера на произвольный узел.

6.6 Производительность

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`WEB_CONCURRENCY`	2 × ядра + 1 (не больше 9)	Число рабочих процессов
`DB_POOL_SIZE`	`5`	Постоянных соединений с базой на процесс
`DB_MAX_OVERFLOW`	`4`	Дополнительных соединений на процесс при пике
`DB_POOL_TIMEOUT`	`30`	Ожидание свободного соединения, с
`DB_CONCURRENCY`	`0` — по числу соединений	Запросов, одновременно работающих с базой, в процессе; остальные ждут в очереди
`DB_QUEUE_TIMEOUT`	`30`	Ожидание в очереди к базе, с; дольше — ответ 503 «Сервер перегружен»
`THREADPOOL_SIZE`	`0` — соединения + 8	Потоков для обработчиков в процессе
`PDF_CONCURRENCY`	`2`	PDF-отчётов, одновременно строящихся в процессе
`RESPONSE_CACHE`	`true`	Кэш ответов аналитики в памяти процесса
`RESPONSE_CACHE_SIZE`	`512`	Ответов в кэше процесса
`GUNICORN_TIMEOUT`	`120`	Тайм-аут запроса, с
`GUNICORN_MAX_REQUESTS`	`2000`	Перезапуск процесса после N запросов (страховка от роста памяти)
`POSTGRES_MAX_CONNECTIONS`	`200`	`max_connections` контейнера PostgreSQL
`MAX_UPLOAD_MB`	`20`	Максимальный размер загружаемого файла

Совет. Соединений с базой нужно не больше `WEB_CONCURRENCY × (DB_POOL_SIZE + DB_MAX_OVERFLOW)` на экземпляр приложения — проверьте, что `max_connections` PostgreSQL с запасом это покрывает. Значения по умолчанию (9 × 9 = 81) укладываются даже в стандартные `max_connections` = 100.

6.7 Журнал

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`LOG_LEVEL`	`INFO`	Уровень журнала
`LOG_JSON`	`false`	Журнал в формате JSON (для Loki, ELK и т. п.); пароли, токены и сессии в журнал не попадают

7 Вход по домену и Active Directory

7.1 Настройка входа по домену

1. Заведите в каталоге служебную учётную запись только для чтения и узнайте базу поиска (`DC=company,DC=local`).
2. Если сертификат контроллера домена выпущен внутренним центром сертификации, положите сертификат центра (PEM) на сервер, например в `/opt/dashboards_serv/certs/company-ca.pem`.
3. Задайте основные переменные в `.env` — сначала с выключенным входом:

```
AUTH_LDAP_ENABLED=false
AUTH_LDAP_URL=ldaps://dc.company.local:636
AUTH_LDAP_BASE_DN=DC=company,DC=local
AUTH_LDAP_BIND_DN=CN=svc-dashboards,OU=Service,DC=company,DC=local
AUTH_LDAP_BIND_PASSWORD=…
```

4. Остальные переменные и сертификат передайте через `docker-compose.override.yml` рядом с `docker-compose.yml`:

```
services:
  app:
    environment:
      AUTH_LDAP_CA_FILE: /certs/company-ca.pem
      AUTH_LDAP_CREATE_USERS: inactive
      AUTH_LDAP_DEFAULT_PROFILE: employee
      AUTH_LDAP_GROUP_PROFILES: "CN=Top
Management,OU=Groups,DC=company,DC=local=executive;CN=Directors,OU=Groups,DC=company,DC=l
ocal=director"
      AUTH_LDAP_ADMIN_GROUP: "CN=Dashboards Admins,OU=Groups,DC=company,DC=local"
      AUTH_LDAP_LOGIN_HINT: "Логин и пароль, как при входе в Windows"
      AD_SYNC_PATH_SOURCE: attributes
    volumes:
      - ./certs:/certs:ro
```

5. `docker compose up -d` и откройте **Администрирование → Вход по домену**: состояние «Настроен, но выключен», замечаний нет, шифрование — LDAPS или StartTLS. Нажмите **Проверить подключение** и найдите несколько сотрудников (**Найти учётную запись**) — каждый должен находиться ровно одной записью и сопоставляться с нужной учётной записью системы.
6. Задайте `AUTH_LDAP_ENABLED=true` и выполните `docker compose up -d`. Проверьте вход тестовым сотрудником.

Подробности о сопоставлении учётных записей, создании при первом входе и профилях по группам — «Руководство администратора», раздел «Вход по домену (LDAP)»; регламент — `docs/runbooks/upgrade-to-0.2.0.md`.

> **Важно.** Учётные записи сопоставляются с каталогом по e-mail. Держите e-mail локальных администраторов вне каталога (или пустым) и сохраните хотя бы одного администратора с паролем системы — он войдёт, даже если каталог недоступен.

7.2 Синхронизация по расписанию

Синхронизация с Active Directory запускается из интерфейса (**Импорт и экспорт → Active Directory**) или командой:

```
docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad                # только план
docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad --apply        # применить
```

ПАРАМЕТР	НАЗНАЧЕНИЕ
`--ad`	Читать сотрудников из Active Directory (`AD_SYNC_*`)
`--csv ФАЙЛ`	Вместо каталога — загрузка списком из файла `.csv` или `.xlsx` (путь внутри контейнера)
`--apply`	Применить план; без него команда только показывает план
`--create-missing`	Создавать сотрудников, которых ещё нет (из AD — со входом по домену)
`--deactivate-missing`	Только AD: отключить пришедших из AD, которых нет в каталоге
`--actor ЛОГИН`	От чьего имени записывать журнал изменений
`--json`	Вывести полный план в JSON

Коды выхода:

КОД	ЗНАЧЕНИЕ
`0`	Готово: план без ошибок (с `--apply` — применён)

КОД	ЗНАЧЕНИЕ
`1`	В плане есть ошибки — ничего не записано; список ошибок — в выводе команды
`2`	Каталог или файл недоступен, не задана настройка или не найден пользователь `--actor`

Пример для cron — каждую ночь в 03:15, вывод — в файл журнала:

```
# /etc/cron.d/dashboards-org-sync
15 3 * * * root cd /opt/dashboards_serv && docker compose exec -T -w /app/backend app python
-m app.cli org-sync --ad --apply --create-missing --actor admin >> /var/log/dashboards-org-
sync.log 2>&1
```

Пример для systemd: служба и таймер.

```
# /etc/systemd/system/dashboards-org-sync.service
[Unit]
Description=Дашборды компании: синхронизация структуры с Active Directory
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
WorkingDirectory=/opt/dashboards_serv
ExecStart=/usr/bin/docker compose exec -T -w /app/backend app python -m app.cli org-sync --
ad --apply --create-missing --actor admin
```

```
# /etc/systemd/system/dashboards-org-sync.timer
[Unit]
Description=Синхронизация с Active Directory каждую ночь

[Timer]
OnCalendar=*-*-* 03:15
Persistent=true

[Install]
WantedBy=timers.target
```

```
systemctl daemon-reload
systemctl enable --now dashboards-org-sync.timer
systemctl list-timers dashboards-org-sync.timer     # когда следующий запуск
journalctl -u dashboards-org-sync.service -n 50     # вывод последнего запуска
```

Код выхода 1 или 2 переводит службу systemd в состояние `failed` (видно в `systemctl --failed`) — подключите его к мониторингу. Перед первым запуском по расписанию посмотрите план без `--apply`, а `--deactivate-missing` включайте, когда убедитесь, что фильтр

`AD_SYNC_USER_FILTER` выбирает всех нужных сотрудников.

8 Входящие данные и интеграции

Версия 0.4.0 добавляет единый путь входящих данных — папку «Входящие» на сервере, загрузку, приём файлов по API с ключом и подключения к системам компании — и отдельный процесс-планировщик. Как ими пользоваться — «Руководство администратора», раздел «Интеграции и входящие данные»; здесь — что нужно на сервере.

8.1 Планировщик (worker)

Планировщик — команда `python -m app.cli worker` в том же образе; в `docker-compose.yml` это сервис `worker`. На каждом такте (`WORKER_TICK_SECONDS`, по умолчанию 30 с) он:

1. разбирает папку «Входящие», если задана `INBOX_DIR`;
2. выполняет запуски подключений, чей срок по расписанию настал;
3. раз в час удаляет исходные файлы обработанных элементов старше `INBOX_RETENTION_DAYS` — из базы и из `processed/`, `rejected/` папки;
4. записывает отметку о работе (`app_settings.worker_status`): время такта, имя сервера, процесс, версию.

Отметку показывает страница **Входящие данные → Папка**, блок «Планировщик»: «Работает» — отметка свежая; «Не отвечает» — отметки нет дольше трёх тактов (не меньше двух минут): процесс остановлен, завис или не видит базу; «Не запущен» — планировщик ни разу не отмечался. HTTP-порта и проверки здоровья у сервиса нет (`healthcheck: disable`), поэтому для мониторинга используйте эту отметку или журнал контейнера.

Свойства сервиса:

- стартует после того, как `app` станет здоровым, и ждёт актуальную схему базы — миграции выполняет только `app` («Планировщик ждёт актуальную схему базы … миграции выполняет сервис app»);
- получает те же переменные окружения, что `app` (якорь `*app-environment` в `docker-compose.yml`), и тот же том `inbox`; переменные, добавленные в override для `app`, добавьте и для `worker`;
- по SIGTERM доделывает текущий такт — разбор файла или запуск подключения — и останавливается (`stop_grace_period: 60s`); долгий запуск подключения, прерванный остановкой, через два часа считается завершённым, и следующий запуск снова возможен;
- **несколько экземпляров** безопасны: запуск по расписанию «занимается» в базе атомарно (`UPDATE … WHERE next_run_at <= now`), а файл папки — атомарным переносом в `.work/`, поэтому одну работу два планировщика не выполнят. Второй экземпляр нужен для отказоустойчивости, а не для скорости: `docker compose up -d --scale worker=2`. Все экземпляры должны видеть один том папки «Входящие».

```
docker compose ps worker                    # Up — без статуса healthy: у сервиса нет
проверки здоровья
docker compose logs -f worker               # «Планировщик запущен: такт 30 с, папка
«Входящие»: /data/inbox»
docker compose exec -w /app/backend worker python -m app.cli worker --once   # один такт
вручную
```

Без планировщика система работает, но папка разбирается только кнопкой «Проверить сейчас», подключения запускаются только кнопкой «Запустить», а очистку по сроку хранения нужно запускать командой (раздел «Без планировщика: cron»).

8.2 Папка «Входящие»

Папка задаётся переменной `INBOX_DIR`; в `docker-compose.yml` это `/data/inbox` в именованном томе `inbox`, общем для `app` и `worker`. В образе каталог `/data/inbox` принадлежит пользователю приложения `app`; подпапки наборов (`hr-months/`, `tp-weekly/`, `indicators/`…), `processed/`, `rejected/` и служебная `.work/` создаются при первом разборе. Устройство папки и правила разбора — «Руководство администратора», раздел «Папка «Входящие»».

Чтобы класть файлы в папку из 1С или с рабочих мест, смонтируйте вместо тома каталог сервера и откройте его по сети (Samba, NFS):

```
# docker-compose.override.yml
services:
  app:
    volumes:
      - /srv/dashboards-inbox:/data/inbox
  worker:
    volumes:
      - /srv/dashboards-inbox:/data/inbox
```

Права: пользователь контейнера (`docker compose exec app id` — UID и GID пользователя `app`) должен читать и записывать каталог и все подпапки — он переносит файлы в `.work/`, `processed/` и `rejected/`. Удобно дать каталогу общую группу с пользователем, от имени которого пишут 1С или сетевая папка, и бит setgid, чтобы новые файлы и подпапки наследовали группу:

```
APP_UID=$(docker compose exec -T app id -u); APP_GID=$(docker compose exec -T app id -g)
sudo mkdir -p /srv/dashboards-inbox
sudo chown -R "$APP_UID:$APP_GID" /srv/dashboards-inbox
sudo chmod -R u+rwX,g+rwX /srv/dashboards-inbox
sudo find /srv/dashboards-inbox -type d -exec chmod g+s {} +
# пользователь сетевой папки или 1С — в группу с GID $APP_GID
```

Без прав на запись страница «Папка» покажет «Только чтение»: файлы читаются, но не переносятся. Каталог должен быть на одном файловом устройстве целиком (перенос в `.work/` — атомарное переименование). Если папка не нужна вовсе, задайте `INBOX_DIR=`

(пусто): страница покажет «Не настроена», остальные способы — загрузка, API, подключения — работают.

Файлы в папке не обязательно резервировать: исходные файлы хранятся и в базе до срока хранения, а элементы, планы и итоги — всегда.

8.3 Сетевой доступ к системам

Проверки и ручные запуски подключений выполняет приложение (`app`), запуски по расписанию — планировщик (`worker`): исходящий доступ нужен обоим контейнерам.

ПОДКЛЮЧЕНИЕ	КУДА	ПОРТ
«По Делу», «Пульс проекта»	адрес системы; для «По Делу» — «Адрес для запросов сервера», если он задан	443 или порт из адреса
Jira, Confluence	сервер Jira / Confluence или `*.atlassian.net`	443
Bitrix24	портал: `*.bitrix24.ru` или коробочный сервер	443
Почтовый ящик: EWS	сервер Exchange (`/EWS/Exchange.asmx`)	443
Почтовый ящик: Microsoft 365	`login.microsoftonline.com`, `graph.microsoft.com`	443
Почтовый ящик: IMAP	почтовый сервер — напрямую, прокси IMAP не использует	993 (SSL/TLS) или 143 (STARTTLS)
1С (OData)	веб-сервер публикации базы	443 или 80

Входящий доступ: 1С, скрипты и другие системы обращаются к приложению по HTTPS через обратный прокси — `POST /api/inbound/v1/files` и `GET /api/inbound/v1/ping`. Размер тела запроса на прокси должен быть не меньше `INBOX_MAX_MB` (в примере nginx — `client_max_body_size 25m`).

Прокси. Запросы к системам по HTTPS идут через `HTTPS_PROXY`, по HTTP — через `HTTP_PROXY`; узлы из `NO_PROXY` — напрямую. Внутренние системы обычно добавляют в `NO_PROXY` (`.company.local`), а прокси оставляют для облачных — Jira Cloud, Microsoft 365, Bitrix24.

Сертификаты. Сертификат системы проверяется по корневым сертификатам образа. Для систем с корпоративным или самоподписанным сертификатом в подключении есть флажок «Проверять сертификат» — выключайте его, только если добавить центр сертификации нельзя.

Разрешённые узлы. На рабочем сервере задайте `<ВИД>_ALLOWED_HOSTS` для используемых подключений (раздел «Входящие данные и интеграции» в «Переменных окружения»): `JIRA_ALLOWED_HOSTS=.company.ru`, `EXCHANGE_ALLOWED_HOSTS=mail.company.ru`. Перенаправления подключения не выполняют, поэтому адрес в подключении должен быть итоговым.

Проверить доступ с сервера:

```
docker compose exec app python -c "import urllib.request;
print(urllib.request.urlopen('https://jira.company.ru/status', timeout=5).status)"
docker compose exec app python -c "import socket;
socket.create_connection(('imap.company.ru', 993), 5); print('ok')"
```

8.4 Секреты подключений и SECRET_KEY

Пароли, токены, вебхуки и секреты приложений подключений хранятся в базе зашифрованными (Fernet: AES-128-CBC и HMAC-SHA256, пакет `cryptography`). Ключ шифрования выводится из `SECRET_KEY` сервера; секрет привязан к адресу, для которого его ввели, и к подключению. В API, журнал изменений, журнал запусков и журнал сервера секреты не попадают: значения секретов, пароль Basic, токен из заголовка Authorization, логин и пароль в адресе и код доступа в адресе вебхука Bitrix24 заменяются на «...».

> **Важно.** `SECRET_KEY` теперь не только «секретная строка» сервера: после его смены сохранённые секреты подключений не расшифровываются. Карточки подключений покажут «Секрет нужно ввести заново», проверки и запуски будут завершаться ошибкой, пока администратор не введёт пароли и токены каждого подключения заново. Перешифрования в 0.4.0 нет. Храните `SECRET_KEY` вместе с резервными копиями базы: копия, восстановленная на сервере с другим ключом, тоже потребует ввести секреты заново.

Если ключ всё же нужно сменить (утечка): подготовьте пароли и токены всех подключений, задайте новый `SECRET_KEY` в `.env`, выполните `docker compose up -d` (перезапустятся `app` и `worker`) и введите секреты в **Администрирование → Интеграции → Настроить** у каждого подключения, затем **Проверить**. Подключения с неработающим секретом видно и командой `docker compose exec -w /app/backend app python -m app.cli integrations list` — в последнем столбце «ошибка: ...».

8.5 Exchange и exchangelib

Режим «Exchange Server (EWS)» почтового ящика работает через пакет `exchangelib`. Он указан в `backend/requirements.txt` и входит в образ. Пакет необязательный: если собираете свой образ без него, удалите строку `exchangelib` из `requirements.txt` — режимы «Microsoft 365 (Graph)» и «IMAP» работают без него, а подключение в режиме EWS сообщит: «Для режима Exchange Server (EWS) на сервере нужен пакет exchangelib — установите его (pip

install exchangelib, он указан в requirements.txt) или выберите режим IMAP или Microsoft Graph». Журнал `exchangelib` не опускается ниже уровня INFO даже при `LOG_LEVEL=DEBUG`: на уровне DEBUG он записывал бы заголовки запросов с паролем.

8.6 Без планировщика: cron

Если отдельный процесс запускать нельзя, удалите или остановите сервис `worker` и поставьте в планировщик ОС команды:

КОМАНДА	ЧТО ДЕЛАЕТ	КОД ВЫХОДА
`python -m app.cli inbox scan`	один разбор папки «Входящие»; итог — JSON	0 — готово, 1 — были ошибки, 2 — папка не настроена или не найдена
`python -m app.cli inbox cleanup`	удалить исходные файлы старше `INBOX_RETENTION_DAYS` — в базе и в папке	0
`python -m app.cli integrations list`	подключения: номер, вид, название, включено ли, расписание, состояние	0
`python -m app.cli integrations run <номер>`	запустить подключение: действия его расписания или первое действие; `--action <ключ>` — конкретное действие	0 — успешно, 1 — частично или ошибка, 2 — подключение не найдено или выключено
`python -m app.cli integrations run <вид>`	то же для всех включённых подключений вида (`jira`, `podelu`…)	то же

Ключи действий: «По Делу» — `sync`, `pull`; «Пульс проекта» — `indicators`; Jira — `refresh`, `preview`; Confluence — `publish`; Bitrix24 — `sales`, `preview`; почтовый ящик — `fetch`; 1С — `load`, `preview`; папка «Входящие» — `scan`.

```
# /etc/cron.d/dashboards-integrations
*/2 * * * * root cd /opt/dashboards_serv && docker compose exec -T -w /app/backend app
python -m app.cli inbox scan >> /var/log/dashboards-inbox.log 2>&1
40 5 * * *  root cd /opt/dashboards_serv && docker compose exec -T -w /app/backend app
python -m app.cli integrations run podelu --action sync >> /var/log/dashboards-
integrations.log 2>&1
*/30 * * * * root cd /opt/dashboards_serv && docker compose exec -T -w /app/backend app
python -m app.cli integrations run 3 >> /var/log/dashboards-integrations.log 2>&1
15 3 * * *  root cd /opt/dashboards_serv && docker compose exec -T -w /app/backend app
python -m app.cli inbox cleanup >> /var/log/dashboards-inbox.log 2>&1
```

Запуск командой записывается в журнал запусков с поводом «Команда» и в журнал изменений; одновременный запуск того же подключения из интерфейса или второй командой не начнётся («Подключение уже выполняет запуск — дождитесь его окончания»). Расписания, заданные в интерфейсе, без планировщика не выполняются — переносите их в cron.

9 База данных и миграции

Схемой базы управляет только Alembic: приложение само таблицы не создаёт и не меняет. Миграции применяются автоматически при старте контейнера; вручную:

```
docker compose exec -w /app/backend app python -m app.cli migrate        # применить
миграции и обновить справочники
docker compose exec -w /app/backend app python -m app.cli check-schema   # код выхода 0 —
схема актуальна
```

> **Важно.** Команды `python -m app.cli` выполняются в каталоге `/app/backend` контейнера — отсюда параметр `-w /app/backend`. Без него команда завершится ошибкой «No module named app».

PostgreSQL рекомендуется для всех установок. SQLite подходит для разработки и небольших стендов: в образе она работает в режиме WAL с проверкой внешних ключей.

МИГРАЦИЯ	ВЕРСИЯ	ЧТО ДЕЛАЕТ
`0012_v0_1_0_schema`	0.1.0	Перестройка всех таблиц с переносом данных прежних версий (откат не поддерживается)
`0013_data_revisions`	0.1.0	Ревизии данных служб для кэша ответов
`0014_org_access`	0.2.0	Структура компании, вход по домену, модель доступа 2.0 (откат поддерживается)
`0015_reports`	0.3.0	Сквозные отчёты: каталог показателей, значения по подразделениям, заметки о статусе, ревизия отчётов (откат поддерживается)
`0016_integrations`	0.4.0	Входящие данные и интеграции: элементы входящих, правила «применять сразу», подключения и журнал запусков, ключи для входящих вызовов (откат поддерживается)

Список таблиц и связей — в «Руководстве администратора» (раздел «Данные и хранение») и в `docs/architecture/adr-0002-database-and-migrations.md`, `adr-0007-org-structure.md`, `adr-0009-access-model.md`, `adr-0010-cross-reports.md`, `adr-0011-integrations.md`.

10 Резервное копирование и восстановление

10.1 PostgreSQL

Резервная копия (без остановки приложения):

```
docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > dashboards-$(date
+%F).dump
```

Восстановление:

```
docker compose stop app worker
docker compose exec -T db pg_restore -U dashboards -d dashboards --clean --if-exists <
dashboards-2026-11-01.dump
docker compose start app worker
```

Копия базы содержит и зашифрованные секреты подключений: восстанавливайте её на сервере с тем же `SECRET_KEY`, иначе секреты придётся ввести заново (раздел «Секреты подключений и SECRET_KEY»). Том папки «Входящие» резервировать не обязательно — исходные файлы хранятся и в базе до срока хранения.

10.2 SQLite

```
docker compose exec app python -c "import sqlite3; s=sqlite3.connect('/data/dashboards.db');
d=sqlite3.connect('/data/backup.db'); s.backup(d)"
docker compose cp app:/data/backup.db ./dashboards-$(date +%F).db
```

> **Совет.** Настройте ежедневное копирование по расписанию (cron, systemd timer) и храните копии минимум 14 дней на другом сервере. Раз в квартал проверяйте восстановление на тестовом стенде.

11 Обновление

11.1 Обычное обновление

```
docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > before-update.dump
git fetch && git checkout vX.Y.Z
docker compose up -d --build
docker compose logs -f app worker
```

Миграции применятся при старте приложения; планировщик дождётся новой схемы и продолжит работу. Откат версии делается восстановлением резервной копии, сделанной перед обновлением.

11.2 Обновление до 0.4.0

Версия 0.4.0 добавляет входящие данные — папку «Входящие» на сервере, загрузку, приём файлов по API с ключом, план и применение — и интеграции с системами компании: «По Делу», «Пульс проекта», Jira, Confluence, Bitrix24, почтовым ящиком Exchange и 1С. Схему меняет миграция `0016_integrations`; она только добавляет таблицы, существующие данные не меняются:

ТАБЛИЦА	СОДЕРЖИМОЕ
`integrations`	подключения: вид, название, настройки без секретов, зашифрованные секреты и адрес их привязки, расписание, состояние, отметки проверки
`integration_runs`	журнал запусков подключений
`inbound_keys`	ключи для входящих вызовов: название, отпечаток SHA-256 и первые 12 символов, области, отзыв
`inbox_items`	элементы входящих: источник, файл (содержимое — до срока хранения), SHA-256, набор, статус, план и итог
`inbox_rules`	правила «применять сразу» для источника и набора

Что меняется в установке:

- в `docker-compose.yml` — новый сервис `worker` (планировщик, тот же образ) и том `inbox` для папки «Входящие», общий для `app` и `worker`; тег образа по умолчанию — `0.4.0`. Если

вы запускаете систему своим compose-файлом, добавьте планировщик и том (раздел «Планировщик (worker)»);
- новые переменные окружения — все необязательные, значения по умолчанию подходят (раздел «Входящие данные и интеграции» в «Переменных окружения»); на рабочем сервере задайте `PUBLIC_URL` и `<ВИД>_ALLOWED_HOSTS` подключений, которые будете использовать, и при необходимости `HTTPS_PROXY`;
- `SECRET_KEY` теперь шифрует секреты подключений — не меняйте его при обновлении и храните вместе с резервными копиями;
- новые зависимости — `requests`, `cryptography`, `exchangelib` — уже в образе;
- страница «Вход по домену» переехала с `/admin/integrations` на `/admin/directory`, прежний адрес открывает «Интеграции».

Пароли, сессии и доступ пользователей не меняются. Новые полномочия «Входящие данные» и «Интеграции» никому не выданы, правила «применять сразу» выключены, подключений и ключей нет. Пункт меню «Входящие данные» сразу появляется у администраторов и у всех, кто вводит данные служб.

Порядок (подробно — `docs/runbooks/upgrade-to-0.4.0.md`):

1. **Сделайте резервную копию** базы: `docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > before-0.4.0.dump` (SQLite — раздел «Резервное копирование и восстановление»).
2. Получите версию и перезапустите систему:

```
git fetch && git checkout v0.4.0
docker compose up -d --build
docker compose logs app | grep "Схема базы"     # Схема базы обновлена: 0015_reports → 0016_integrations
docker compose logs worker | grep "Планировщик"  # Планировщик запущен: такт 30 с, папка «Входящие»: /data/inbox
```

3. Проверьте `/api/health` (`"version": "0.4.0"`, `schema.current == schema.head == "0016_integrations"`), войдите администратором: в меню — «Входящие данные», в «Администрировании» — «Интеграции» и «Вход по домену»; на странице **Входящие данные → Папка** — папка «Работает», планировщик «Работает».
4. Задайте разрешённые узлы и `PUBLIC_URL`, выпустите ключи, подключите системы и включите «применять сразу» для проверенных источников («Руководство администратора», раздел «Интеграции и входящие данные»).

Откат. Миграция 0016 поддерживает откат: удаляются таблицы входящих данных и интеграций, а полномочия «Входящие данные» и «Интеграции» снимаются с пользователей (в 0.3.0 их нет). Данные, которые входящие и подключения уже записали, — значения показателей, метрики, недели отчёта, структура и сотрудники — остаются: это данные 0.3.0.

Предпочтительно восстановить резервную копию, сделанную перед обновлением, и вернуть 0.3.0; планировщик останавливается и удаляется заранее — в `docker-compose.yml` версии 0.3.0 его нет:

```
docker compose stop app worker && docker compose rm -f worker
docker compose exec -T db pg_restore -U dashboards -d dashboards --clean --if-exists <
before-0.4.0.dump
git checkout v0.3.0 && docker compose up -d --build --remove-orphans
```

Если после обновления внесены данные, которые нужно сохранить, откатите схему образом 0.4.0, затем верните 0.3.0:

```
docker compose stop app worker && docker compose rm -f worker
docker compose run --rm --no-deps -w /app/backend --entrypoint alembic app downgrade
0015_reports
git checkout v0.3.0 && docker compose up -d --build --remove-orphans
```

При откате схемы теряются элементы входящих с исходными файлами, правила, подключения с секретами и журналом запусков и ключи для входящих вызовов: системы, которые присылали файлы по ключу, начнут получать ответ 404. Том `inbox` остаётся — удалите его, если он не нужен: `docker volume rm dashboards_serv_inbox`.

11.3 Обновление до 0.3.0

Версия 0.3.0 добавляет сквозной отчёт — статус компании и подразделений по показателям с учётом уровня доступа сотрудника, ввод показателей и их каталог. Схему меняет миграция `0015_reports`; она только добавляет таблицы, существующие данные не меняются:

ТАБЛИЦА	СОДЕРЖИМОЕ
`indicators`	каталог показателей: класс данных, направление, свёртка, правило светофора, источник, служба-владелец, где ожидается значение
`indicator_values`	введённые значения, цели и комментарии по подразделениям за месяц
`unit_status_notes`	заметки руководителей «под контролем» за месяц
`report_revision`	ревизия отчётов — по ней обновляется кэш расчёта

Настраивать ничего не нужно: новых переменных окружения нет, в `docker-compose.yml` изменился только тег образа по умолчанию (`0.3.0`). Пароли, сессии и доступ пользователей не меняются; новое полномочие «Ведёт сквозные отчёты» никому не выдано. При первом

запуске каталог показателей получает восемь показателей, вычисляемых из данных служб (приём и увольнение, текучесть, время закрытия позиции, заявки в работе и решённые, решено/получено, доля партнёрств в норме): отчёт сразу показывает их; вводимые показатели (финансовые, операционные) администратор добавляет сам. Глубина детализации и классы данных из профилей доступа начинают действовать в сквозном отчёте: проверьте профили, прежде чем объявлять отчёт сотрудникам.

Порядок (подробно — `docs/runbooks/upgrade-to-0.3.0.md`):

1. **Сделайте резервную копию** базы: `docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > before-0.3.0.dump` (SQLite — раздел «Резервное копирование и восстановление»).
2. Получите версию и перезапустите приложение:

```
git fetch && git checkout v0.3.0
docker compose up -d --build
docker compose logs app | grep "Схема базы"     # Схема базы обновлена: 0014_org_access → 0015_reports
```

3. Проверьте `/api/health` (`"version": "0.3.0"`, `schema.current == schema.head == "0015_reports"`), войдите администратором: в меню — «Сквозной отчёт», в «Администрировании» — «Показатели» с восемью вычисляемыми показателями.
4. Заполните каталог показателей и выдайте права на их ввод («Руководство администратора», раздел «Показатели и сквозной отчёт»).

Откат. Миграция 0015 поддерживает откат: удаляются таблицы отчётов и настройка «Подразделения служб», остальные данные не затрагиваются. Предпочтительно восстановить резервную копию, сделанную перед обновлением, и вернуть 0.2.0:

```
docker compose stop app
docker compose exec -T db pg_restore -U dashboards -d dashboards --clean --if-exists < before-0.3.0.dump
git checkout v0.2.0 && docker compose up -d --build
```

Если после обновления внесены данные служб, которые нужно сохранить, откатите схему образом 0.3.0, затем верните 0.2.0:

```
docker compose stop app
docker compose run --rm --no-deps -w /app/backend --entrypoint alembic app downgrade 0014_org_access
git checkout v0.2.0 && docker compose up -d --build
```

При откате схемы теряются каталог показателей, введённые значения, заметки «под контролем» и подразделения служб. Полномочие «Ведёт сквозные отчёты» откат у пользователей не снимает — снимите его до отката (вкладка «Полномочия» карточки

пользователя): версия 0.2.0 такого полномочия не знает и не даст сохранить другие полномочия этих пользователей.

11.4 Обновление до 0.2.0

Версия 0.2.0 добавляет структуру компании, профили доступа, вход по учётной записи домена и синхронизацию с Active Directory. Схему меняет миграция `0014_org_access`:

ТАБЛИЦА	ИЗМЕНЕНИЕ
`departments`	становится деревом: родитель, тип уровня, порядок, внешний идентификатор и источник; имена уникальны среди соседних, код необязателен
`department_heads`	новая: руководители подразделений с основным
`users`	основное подразделение, явный руководитель, «первое лицо», способ входа и связь с каталогом, профиль доступа, личные настройки, полномочия; пароль может быть пустым (доменная учётная запись, «без доступа»)
`user_departments` → `user_access_units`	первое по алфавиту подразделение пользователя становится основным, остальные — дополнительными подразделениями области доступа
`service_access`	уровень «нет доступа»; вместо признака «только свои подразделения» — область службы (`department_scope = true` → «Своя ветка»)
`access_profiles`, `app_settings`, `org_revision`	новые: профили доступа (встроенные создаются при старте), настройки структуры, ревизия структуры

Доступ существующих пользователей сохраняется: профиль им не назначается («Индивидуальная настройка» — вся компания, все классы данных, реестры и выгрузка, как в 0.1.0), уровни служб остаются прежними. Пароли и сессии не меняются: повторно входить не нужно. Подразделения 0.1.0 становятся верхним уровнем дерева — вложить их в нужные уровни можно после обновления.

Порядок (подробно — `docs/runbooks/upgrade-to-0.2.0.md`):

1. **Сделайте резервную копию** базы (раздел «Резервное копирование и восстановление»).
2. Получите версию и, если нужно, добавьте в `.env` новые переменные (`TIMEZONE`; вход по домену — позже):

```
git fetch && git checkout v0.2.0
docker compose up -d --build
docker compose logs app | grep "Схема базы"     # Схема базы обновлена:
0013_data_revisions → 0014_org_access
```

3. Проверьте `/api/health` (`"version": "0.2.0"`, `schema.current == schema.head`), войдите администратором, откройте «Пользователи» (у всех — «Индивидуальная настройка» и прежние службы), «Подразделения» и «Структуру компании».
4. Постройте структуру, назначьте профили доступа и при необходимости включите вход по домену («Руководство администратора»).

Откат. Миграция 0014 поддерживает откат, но лучший способ — восстановить резервную копию, сделанную перед обновлением, и вернуть прежнюю версию:

```
docker compose stop app
docker compose exec -T db pg_restore -U dashboards -d dashboards --clean --if-exists <
before-0.2.0.dump
git checkout v0.1.0 && docker compose up -d --build
```

Если после обновления уже внесены данные, которые нужно сохранить, откатите схему образом 0.2.0 (до переключения на 0.1.0):

```
docker compose stop app
docker compose run --rm --no-deps -w /app/backend --entrypoint alembic app downgrade
0013_data_revisions
git checkout v0.1.0 && docker compose up -d --build
```

При откате схемы теряется то, чего нет в 0.1.0: дерево (все подразделения становятся плоским списком, повторяющиеся названия получают номер, пустые коды — код вида `D12`), руководители подразделений, профили доступа и личные настройки, уровни «нет доступа», полномочия и привязка к домену. Доменные учётные записи и учётные записи «без доступа» после отката не могут войти — выдайте им пароль командой `reset-password`.

11.5 Обновление до 0.1.0 с версий до 0.1.0

Версия 0.1.0 — первая версия продукта; прежние версии («Dashboards API 1.1») хранили данные в SQLite в томе `db_data`. Миграция `0012_v0_1_0_schema` перестраивает все таблицы с переносом данных; дальше при том же старте применяются 0013–0016. Установки до 0.1.0 обновляются сразу до 0.4.0.

1. **Сделайте копию тома** со старой базой:

```
docker run --rm -v dashboards_serv_db_data:/data -v "$PWD":/backup alpine cp
/data/hr_dashboard.db /backup/hr_dashboard-before-0.1.0.db
```

2. **Вариант А — остаться на SQLite.** В `.env` задайте `DATABASE_URL=sqlite:////data/hr_dashboard.db` и подключите старый том к сервису `app` (раздел `volumes: - db_data:/data`). При старте база будет обновлена на месте.
3. **Вариант Б — перейти на PostgreSQL (рекомендуется).** Обновите копию старой базы, затем перенесите данные:

```
# обновить копию SQLite до актуальной схемы
docker compose run --rm --no-deps -w /app/backend --entrypoint python -v "$PWD":/backup -
e DATABASE_URL=sqlite:////backup/hr_dashboard-before-0.1.0.db app -m app.cli migrate
# создать схему в PostgreSQL и перенести данные
docker compose up -d db
docker compose run --rm -w /app/backend --entrypoint python app -m app.cli migrate
docker compose run --rm -w /app/backend --entrypoint python -v "$PWD":/backup app
scripts/sqlite_to_postgres.py /backup/hr_dashboard-before-0.1.0.db
docker compose up -d
```

4. **Проверьте** после входа: месяцы службы персонала, недели отчёта ТП, число заявок Naumen и партнёрств на странице «О системе».

Что меняется для пользователей при обновлении с версий до 0.1.0:

- роли «Начальник службы персонала» и «Просмотр отдела» превращаются в доступ к службе персонала: «Управление» и «Просмотр» своей ветки структуры соответственно; роль «Просмотр» — в обычную учётную запись с её прежним доступом к службам;
- уровни доступа «чтение», «правка», «метрики», «админ» превращаются в «Просмотр», «Ввод данных», «Ввод данных», «Управление»;
- вход выполняется заново: прежние токены недействительны.

Подробности и проверки — `docs/runbooks/upgrade-to-0.1.0.md`.

12 Производительность и масштабирование

Конфигурация по умолчанию (4 ядра, `WEB_CONCURRENCY=9`, PostgreSQL в соседнем контейнере) рассчитана на 500–1000 одновременно работающих пользователей. Нагрузочная проверка 0.3.0 со сквозным отчётом — `docs/performance/load-test-0.3.0.md`; проверка 0.2.0: 1000 одновременных пользователей всех уровней доступа на 4 ядрах — p50 16 мс, p95 53 мс без ошибок сервера, структура компании (`/api/org/tree`) — p95 32 мс (`docs/performance/load-test-0.2.0.md`; методика и рекомендации по массовому входу — `load-test-0.1.0.md`); сценарий нагрузки — `loadtest/`.

Рекомендации:

- держите PostgreSQL на SSD; для больших объёмов заявок Naumen увеличьте `shared_buffers` до 25% памяти;
- при росте нагрузки сначала увеличивайте число процессов (`WEB_CONCURRENCY`), затем — число экземпляров приложения за балансировщиком; сессии хранятся в базе, «липкие» сессии не нужны;
- PDF-отчёты — самая тяжёлая операция (генерация графиков); при массовом формировании отчётов добавьте процессы;
- структура компании (`/api/org/tree`) строится один раз на ревизию структуры в каждом процессе: частые изменения структуры (например, синхронизация с AD каждые несколько минут) сбрасывают этот кэш — синхронизируйте раз в сутки или реже;
- сквозной отчёт за месяц рассчитывается один раз на ревизии структуры, данных служб и значений показателей и дальше только отбирается под права пользователя; ввод значений и загрузка данных служб сбрасывают этот расчёт — первый запрос после изменения медленнее остальных;
- проверка пароля в каталоге занимает время сети: держите `AUTH_LDAP_TIMEOUT` небольшим (5 с), чтобы недоступный каталог не задерживал вход надолго;
- приём файлов по API, разбор папки и запуски подключений строят план и применяют данные тем же движком, что ручная загрузка, и так же сбрасывают кэш ответов и расчёт сквозного отчёта: не ставьте выгрузки и подключения чаще, чем данные действительно меняются (обычно — раз в 15–60 минут); подключения Jira, Bitrix24 и 1С ничего не записывают, если значения не изменились;
- ручной запуск подключения выполняется в процессе приложения и держит его поток до конца запуска; долгие запуски (Bitrix24 с тысячами сделок, 1С с большими выборками) ставьте в расписание — их выполняет планировщик, отдельно от запросов пользователей;
- частоту приёма по API на один ключ ограничивает `INBOUND_RATE_LIMIT` (120 запросов в минуту); счётчик ведёт каждый процесс приложения отдельно, поэтому при нескольких процессах и экземплярах фактический предел выше.

13 Обслуживание

Команды выполняются из каталога установки (`/opt/dashboards_serv`) на сервере.

ЗАДАЧА	КОМАНДА
Сбросить пароль пользователя	`docker compose exec -w /app/backend app python -m app.cli reset-password <логин>`
Создать администратора	`docker compose exec -w /app/backend -e ADMIN_PASSWORD='…' app python -m app.cli create-admin <логин>`
Удалить истёкшие сессии	`docker compose exec -w /app/backend app python -m app.cli cleanup-sessions`
Загрузить демо-данные в пустые разделы	`docker compose exec -w /app/backend app python -m app.cli demo-data`
Проверить схему базы	`docker compose exec -w /app/backend app python -m app.cli check-schema`
Применить миграции вручную	`docker compose exec -w /app/backend app python -m app.cli migrate`
План синхронизации с Active Directory	`docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad`
Синхронизировать с Active Directory	`docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad --apply [--create-missing] [--deactivate-missing]`
Загрузить список сотрудников из файла	`docker compose cp список.csv app:/tmp/list.csv` , затем `docker compose exec -w /app/backend app python -m app.cli org-sync --csv /tmp/list.csv --apply --create-missing`
Резервная копия PostgreSQL	`docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > dashboards-$(date +%F).dump`
Разобрать папку «Входящие» один раз	`docker compose exec -T -w /app/backend app python -m app.cli inbox scan`
Удалить исходные файлы входящих старше срока хранения	`docker compose exec -T -w /app/backend app python -m app.cli inbox cleanup`

ЗАДАЧА	КОМАНДА
Список подключений и их состояние	`docker compose exec -T -w /app/backend app python -m app.cli integrations list`
Запустить подключение	`docker compose exec -T -w /app/backend app python -m app.cli integrations run <номер или вид> [--action <действие>]`
Один такт планировщика	`docker compose exec -T -w /app/backend worker python -m app.cli worker --once`
Журнал планировщика	`docker compose logs -f worker`
Перезапустить планировщик	`docker compose restart worker`

Истёкшие сессии не мешают работе (сервер их не принимает), но очистку раз в сутки удобно поставить в расписание вместе с резервным копированием и синхронизацией с AD. Разбор папки, запуски подключений и очистку входящих по сроку хранения выполняет планировщик — ставить их в cron нужно, только если сервис `worker` не запущен (раздел «Без планировщика: cron»).

14 Диагностика

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
Контейнер `app` перезапускается, в журнале «SECRET_KEY не задан»	Задайте `SECRET_KEY` не короче 32 символов в `.env`
Контейнеры `app` и `worker` перезапускаются, в журнале «PUBLIC_URL — без «/» в конце...» или «PUBLIC_URL — адрес http(s)://...»	Исправьте `PUBLIC_URL` в `.env`: `https://dashboards.company.ru`, без «/» в конце, или оставьте пустым
`/api/health` отвечает `degraded`	Схема базы не актуальна: `docker compose exec -w /app/backend app python -m app.cli migrate`, смотрите журнал
«No module named app» при запуске `python -m app.cli`	Команда выполнена не в `/app/backend`: добавьте `-w /app/backend`
«Сессия устарела: обновите страницу»	Не совпал CSRF-токен (например, после выхода в другой вкладке) — обновите страницу
Вход сразу «выбрасывает» за HTTPS-прокси	Включён `SESSION_COOKIE_SECURE=true`, а сайт открыт по HTTP — откройте по HTTPS или выключите параметр
«Слишком много неудачных попыток»	Сработало ограничение частоты входа; подождите или перезапустите приложение
«Сервер домена не отвечает — попробуйте войти позже»	Каталог недоступен с сервера приложения: проверьте адрес, порт, межсетевой экран и сертификат на странице «Вход по домену» (**Проверить подключение**); локальные учётные записи входят по паролю системы
«Каталог не принял служебную учётную запись»	Неверные `AUTH_LDAP_BIND_DN` или `AUTH_LDAP_BIND_PASSWORD`, учётная запись заблокирована
Ошибка сертификата при подключении к каталогу	Укажите `AUTH_LDAP_CA_FILE` (сертификат центра сертификации домена) и смонтируйте файл в контейнер
«Учётной записи в системе нет» у сотрудника домена	Нет учётной записи с его e-mail: создайте её, загрузите сотрудников или включите `AUTH_LDAP_CREATE_USERS`
Сотрудник домена не находится или находится дважды	Проверьте `AUTH_LDAP_USER_FILTER` на странице «Вход по домену» (**Найти учётную запись**): нужна ровно одна запись

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
`org-sync` завершается с кодом 2	Каталог недоступен или не заданы `AD_SYNC_*` / `AUTH_LDAP_URL` и `AUTH_LDAP_BASE_DN`
`org-sync` завершается с кодом 1	В плане ошибки (например, цикл подчинения, повтор e-mail): запустите без `--apply` и с `--json`, исправьте данные в каталоге
Записи журнала за вечер попадают в следующий день, время в PDF сдвинуто	Задайте `TIMEZONE` часового пояса компании
Ошибка 413 при загрузке файла	Увеличьте `MAX_UPLOAD_MB` и `client_max_body_size` прокси
Медленные страницы при большом числе пользователей	Увеличьте `WEB_CONCURRENCY`, проверьте `max_connections` PostgreSQL и загрузку процессора
Сквозной отчёт: «Нет доступных показателей», на «Обзоре» нет карточки отчёта	В каталоге нет активных показателей классов, доступных пользователю (или они выключены): проверьте «Администрирование → Показатели»
Сквозной отчёт: «Область отчёта пуста»	У пользователя нет основного, руководимых и дополнительных подразделений при области «Своя ветка» или «Своё подразделение» — укажите подразделение в карточке пользователя
Показатели техподдержки или партнёрств не попадают в сквозной отчёт	Не найдено подразделение службы (по коду TECH, PROJ): «Показатели → Подразделения служб»
«Входящие данные → Папка»: планировщик «Не запущен» или «Не отвечает»	Сервис `worker` не запущен, остановлен или не видит базу: `docker compose ps worker`, `docker compose logs worker`. «Планировщик ждёт актуальную схему базы» — приложение ещё не применило миграции
Папка «Входящие» — «Не настроена», «Не найдена» или «Только чтение»	Не задана `INBOX_DIR`, каталог не смонтирован в контейнер или у пользователя контейнера нет прав на запись (раздел «Папка «Входящие»»)
Файлы лежат в папке «Входящие» и не забираются	Планировщик не работает; файл ещё меняется (меньше `INBOX_SETTLE_SECONDS`); имя скрытое или временное (`~$*`, `*.tmp`, `*.part`); файл в подпапке, имя которой не совпадает с набором; файл — символьная ссылка

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
На карточке подключения «Секрет нужно ввести заново»	Сменился `SECRET_KEY` или база восстановлена на сервере с другим ключом — введите секреты заново (раздел «Секреты подключений и SECRET_KEY»)
«Подключение к … запрещено настройками сервера (…_ALLOWED_HOSTS)»	Добавьте узел в переменную и перезапустите `app` и `worker`: `docker compose up -d`
«Не удалось подключиться к …», «Превышено время ожидания подключения к …» при проверке подключения	Нет сетевого доступа из контейнеров к системе: межсетевой экран, `HTTPS_PROXY` и `NO_PROXY` (раздел «Сетевой доступ к системам»)
1С или скрипт получает 401 или 429 от `/api/inbound/v1`	401 — ключ неверен, отозван или выпущен заново (`invalid_key`) либо нет заголовка `Authorization` (`auth_required`); 429 — больше `INBOUND_RATE_LIMIT` запросов в минуту, повторите через `Retry-After` секунд
«Для режима Exchange Server (EWS) на сервере нужен пакет exchangelib»	Образ собран без `exchangelib` — верните пакет в `requirements.txt` или используйте IMAP или Microsoft Graph
Карточка подключения долго показывает «Выполняется запуск»	Процесс остановили посреди запуска; через два часа после начала запуск снова возможен — запустите подключение по расписанию или командой `integrations run`

Журнал приложения: `docker compose logs app`, планировщика — `docker compose logs worker`. У каждого ответа API есть заголовок `X-Request-ID` — по нему запрос находится в журнале. Подробный разбор неполадок интеграций и входящих данных — `docs/runbooks/integrations-troubleshooting.md`.